

Generative Models

CMSC 178IP - Digital Image Processing

Noel Jeffrey Pinton

University of the Philippines - Cebu
Department of Computer Science

Course Outline

Introduction to Generative Models

Autoencoders: The Foundation

Variational Autoencoders (VAE)

Generative Adversarial Networks (GAN)

VAE vs GAN Comparison

Applications

Best Practices

Summary

Introduction to Generative Models

What are Generative Models?

Simple Definition

Generative models learn to create new, realistic data that looks like it came from the training dataset.

Everyday Analogy:

- **Artist Learning to Paint:** Watch many paintings → Learn the style → Create new original paintings
- **Chef Creating Recipes:** Taste many dishes → Learn flavor patterns → Invent new recipes

In Machine Learning:

- Learn from real images
- Understand patterns and structure
- Generate new, never-seen-before images

Why Do We Need Generative Models?

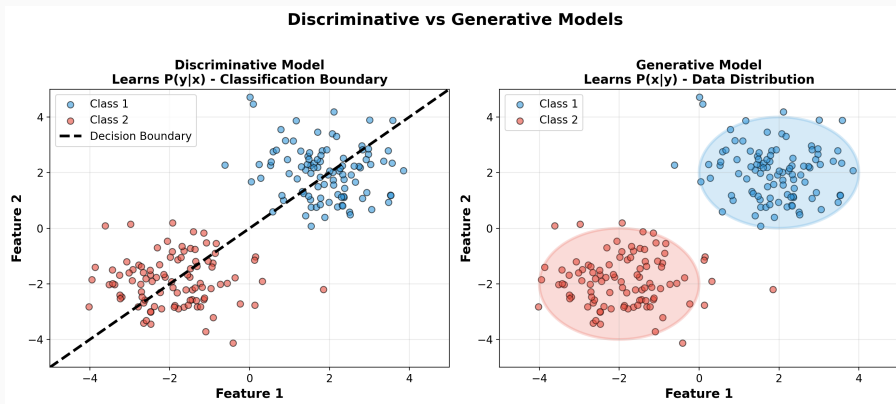
Key Applications:

- **Create new content:** Art, faces, scenes
- **Data augmentation:** More training examples
- **Understand data:** Learn what makes images realistic
- **Fill in missing info:** Complete partial images
- **Transform images:** Style transfer, colorization

Real-World Impact:

- Medical imaging (synthetic training data)
- Entertainment (movie effects, game assets)
- Privacy (synthetic faces instead of real ones)

Discriminative vs Generative Models



Key Difference: Discriminative models classify, generative models create

Types of Generative Models

We'll Cover Two Main Approaches: Variational Autoencoders (VAE):

- Learn compressed representation
- Probabilistic approach
- Stable training
- Slightly blurry outputs

Generative Adversarial Networks (GAN):

- Two networks compete
- Adversarial training
- Sharp, realistic outputs
- Can be tricky to train

Both approaches: Learn from data → Generate new samples

Autoencoders: The Foundation

What is an Autoencoder?

Simple Idea

Compress an image into a small code, then reconstruct it back. Like a smart ZIP file that learns compression!

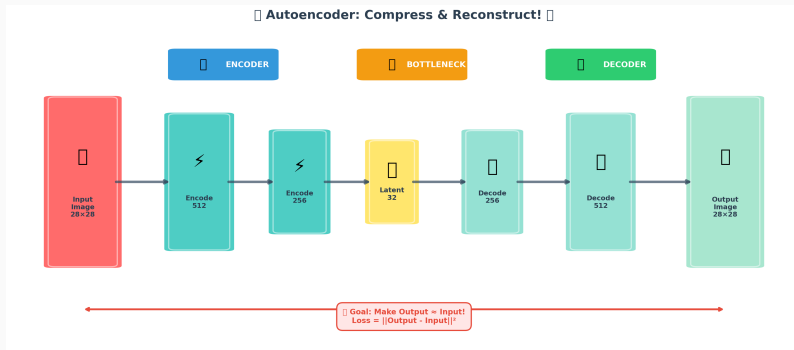
Two Parts:

- **Encoder:** Compress image → Small code (like squeezing a sponge)
- **Decoder:** Small code → Reconstructed image (release the sponge)

Why Useful?

- Forces network to learn important features
- Throws away noise and details
- Creates useful compressed representation

Autoencoder Architecture



Goal: Make output look like input (minimize reconstruction error)

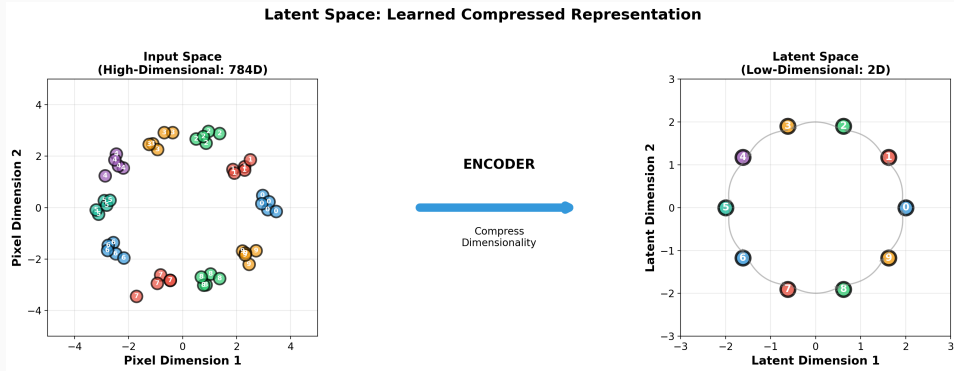
What is Latent Space?

The "compressed code" in the middle where similar images are close together. Think of it as a map of all possible images!

Analogy:

- Full image = Street address (detailed, long)
- Latent code = GPS coordinates (compact, essential info)
- Nearby coordinates = Similar locations

Latent Space Visualization



Compression from 784 dimensions (28×28) to just 2 dimensions

Problem with Basic Autoencoders

Issue: Can't generate NEW images easily

- Learns to compress and reconstruct
- Latent space has "gaps" and irregularities
- Random sampling gives bad results

Solution: Make latent space smooth and continuous → **Variational Autoencoder (VAE)**

Variational Autoencoders (VAE)

What Makes VAE Different?

Key Insight

Instead of encoding to a single point, encode to a probability distribution. This creates a smooth, continuous latent space perfect for generation!

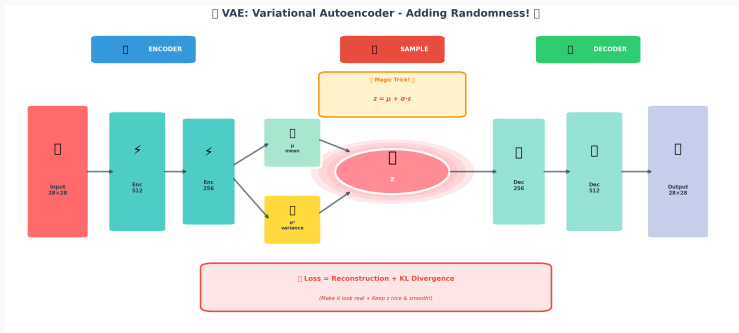
Regular Autoencoder:

- Image $\rightarrow$ Single code (one point)
- Gaps in latent space

VAE:

- Image $\rightarrow$ Distribution of codes (cloud of points)
- Smooth, continuous latent space
- Can sample new codes easily

VAE Architecture



Mean (μ) and variance (σ^2) instead of single code

How VAE Encoding Works

Step-by-Step:

1. Encoder produces mean (μ) and variance (σ^2)
2. Sample code: $z = \mu + \sigma \cdot \epsilon$ where $\epsilon \sim N(0, 1)$
3. Decoder reconstructs from sampled code

Why This Helps:

- Forces smooth latent space
- Similar images have overlapping distributions

VAE Loss Function (Simplified)

Two Goals:

1. **Reconstruction:** Output should look like input

- $\text{Loss}_{\text{recon}} = ||\text{input} - \text{output}||^2$

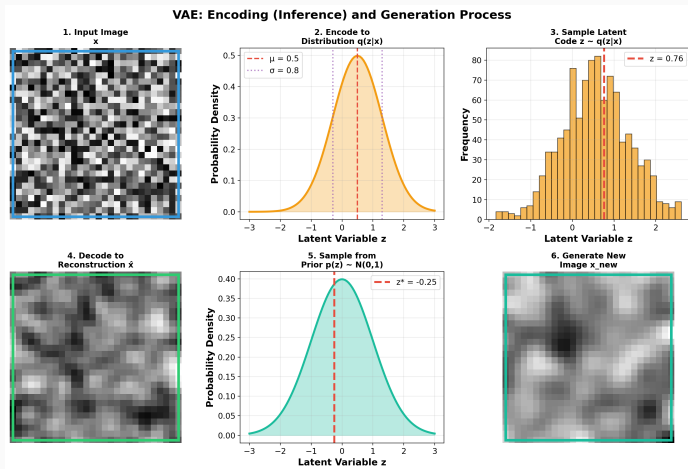
2. **Regularization:** Keep distributions close to standard normal

- KL Divergence: Push μ toward 0, σ toward 1
- Prevents "cheating"

Total Loss = Reconstruction + KL Divergence

Balance is key!

VAE Sampling Process



Top: Encoding. Bottom: Generation

Two Ways to Generate:

1. Reconstruction:

- Encode real image
- Sample from learned distribution
- Decode to get similar image

2. Pure Generation:

- Sample random code from $N(0, 1)$
- Decode to get completely new image
- Works because latent space is smooth!

Advantages:

- Stable and easy to train
- Smooth latent space
- Principled probabilistic framework
- Good for interpolation

Limitations:

- Outputs tend to be blurry
- Less sharp than real images
- Trade-off between quality and smoothness

Best for: Understanding data structure, interpolation, compression

Generative Adversarial Networks (GAN)

GAN: A Different Approach

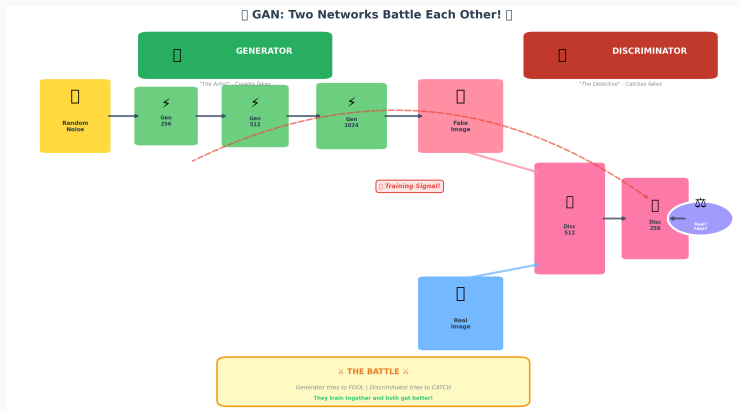
Core Idea

Two networks compete: Generator tries to create fake images, Discriminator tries to spot fakes. Competition makes both better!

Analogy: Art Forger vs Art Detective

- **Forger (Generator):** Creates fake paintings, improves to fool detective
- **Detective (Discriminator):** Learns to spot fakes, gets better over time
- **Result:** Forger becomes so good, detective can't tell real from fake!

GAN Architecture



Two networks trained simultaneously in competition

How GAN Training Works

Training Loop:

1. Train Discriminator:

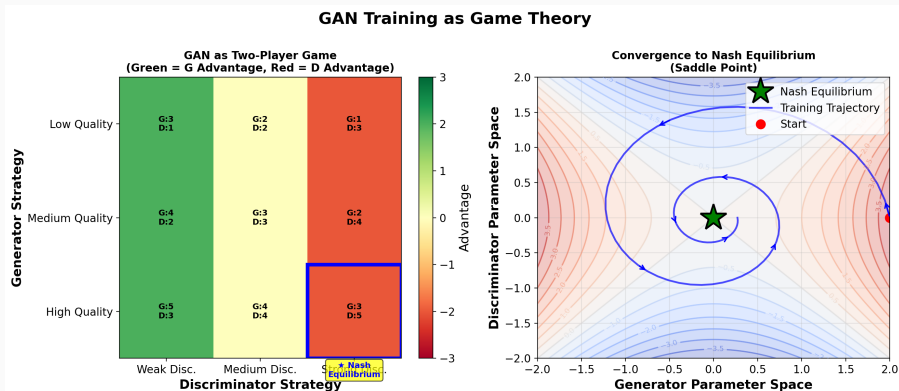
- Show real images $\rightarrow$ Label "real" (1)
- Show fake images from generator $\rightarrow$ Label "fake" (0)
- Update D to get better at classification

2. Train Generator:

- Generate fake images
- Try to fool discriminator
- Update G based on discriminator's response

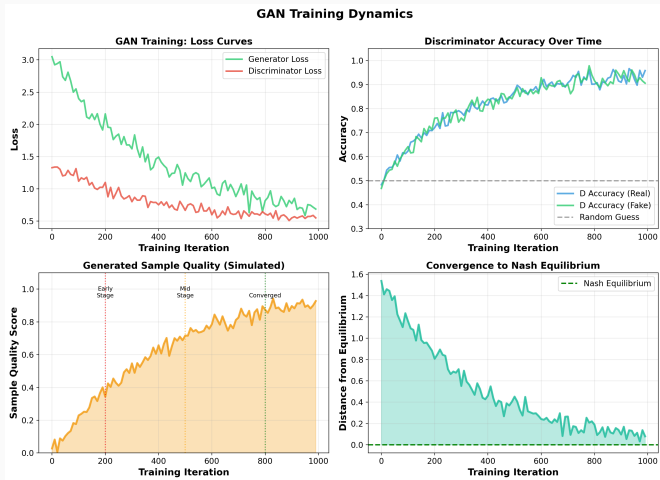
Repeat: Alternate between training D and G

GAN as a Game



Goal: Reach Nash equilibrium where neither can improve

GAN Training Dynamics



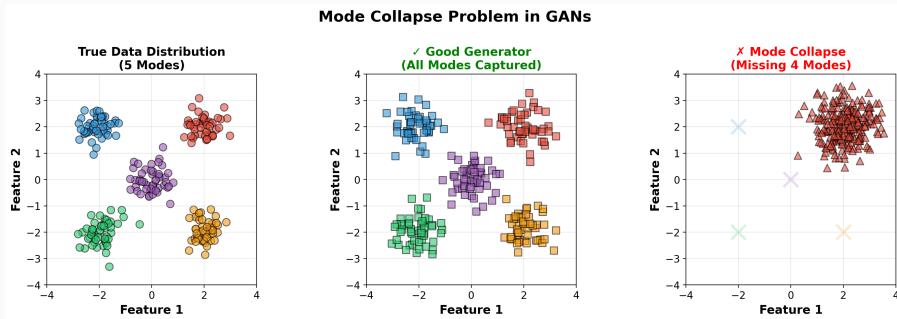
1. Training Instability:

- Hard to balance G and D
- One can dominate the other
- Requires careful tuning

2. Mode Collapse:

- Generator produces limited variety
- Ignores some modes of data
- Repetitive outputs

Mode Collapse Problem



Generator fails to capture full diversity of data

3. Vanishing Gradients:

- If D is too good, G gets no useful feedback
- Learning stops
- Need to carefully balance

4. Evaluation Difficulty:

- No single metric for quality
- Visual inspection needed
- Inception Score and FID commonly used

Best Practices:

- Use batch normalization
- Add label smoothing (0.9 not 1.0)
- Add noise to discriminator inputs
- Use Adam optimizer, low learning rate
- Monitor both losses

Key Insight: Balance is everything!

GAN Pros and Cons

Advantages:

- Sharp, realistic outputs
- High quality generations
- No explicit likelihood needed
- Creative applications

Limitations:

- Difficult to train
- Mode collapse issues
- No probabilistic interpretation
- Requires careful tuning

Best for: High-quality image generation, realistic samples

VAE vs GAN Comparison

VAE vs GAN: Side by Side

VAE vs GAN: Comprehensive Comparison

Variational Autoencoder (VAE)

TRAINING OBJECTIVE

Maximize ELBO

$$\mathcal{L} = \mathbb{E}[\log p(x|z)] - D_{KL}[q(z|x) \| p(z)]$$

- ✓ Explicit likelihood
- ✓ Stable training
- ✓ Structured latent space
- ✗ Blurry/smooth outputs

CHARACTERISTICS

Training Stability	★★★★★
Output Quality	★★★☆☆
Mode Coverage	★★★★☆
Latent Space	★★★★★
Speed	★★★★☆

Generative Adversarial Network (GAN)

TRAINING OBJECTIVE

Minimax Game

$$\min_G \max_D V(D, G) = \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

- ✓ Sharp, realistic outputs
- ✓ High sample quality
- ✗ Training instability
- ✗ Mode collapse risk

CHARACTERISTICS

Training Stability	★★★☆☆
Output Quality	★★★★★
Mode Coverage	★★★☆☆
Latent Space	★★★☆☆
Speed	★★★☆☆

When to Use Which?

Choose VAE when:

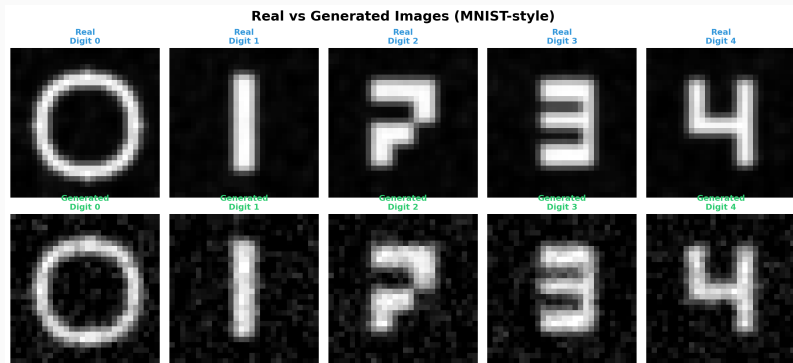
- Need stable training
- Want smooth latent space
- Interpolation is important
- Understanding data structure is goal
- Probabilistic framework needed

Choose GAN when:

- Image quality is critical
- Have time for tuning
- Realistic samples needed
- Don't need latent space interpretation

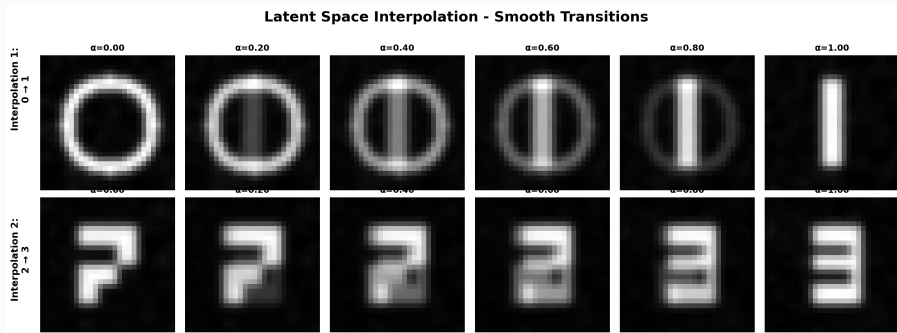
Applications

Generation Examples



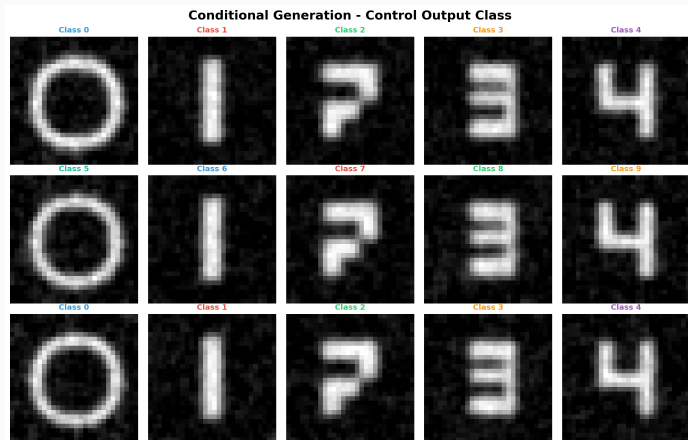
Real vs generated MNIST-style digits

Latent Space Interpolation



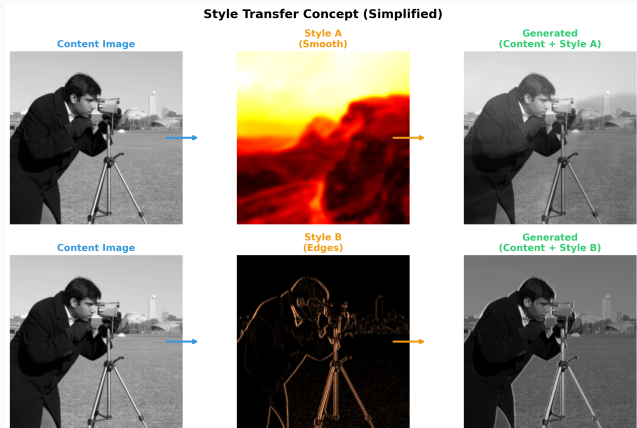
Smooth transitions between digits in latent space

Conditional Generation



Generate specific classes on demand

Style Transfer



Combine content and style

Real-World Applications

Generative Models: Real-World Applications



Image Generation

- Generate realistic images
- MNIST, CIFAR, faces
- Art creation



Image-to-Image

- Style transfer
- Colorization
- Super-resolution



Data Augmentation

- Synthetic training data
- Rare class generation
- Domain adaptation



Anomaly Detection

- Reconstruction error
- Out-of-distribution
- Quality control



Text-to-Image

- Generate from caption
- Conditional synthesis
- Creative tools



Medical Imaging

- Synthetic MR/CT
- Data privacy
- Rare disease simulation

Use Case: Synthetic medical images for training

Benefits:

- Privacy: No real patient data needed
- Augmentation: Generate rare conditions
- Accessibility: More training data available

Approach:

- Train on real MRI/CT scans
- Generate synthetic but realistic scans
- Use for training diagnostic models
- Validate on real data

Problem: Limited training data

Solution: Generate synthetic examples

- Train generative model on available data
- Generate new variations
- Augment training set
- Improve model performance

Especially Useful For:

- Rare classes
- Expensive data collection
- Privacy-sensitive domains

Best Practices

Training Tips and Best Practices

VAE Training Tips

- ✓ Use β -VAE for better disentanglement
- ✓ Balance reconstruction vs KL term
- ✓ Warm-up KL weight gradually
- ✓ Monitor ELBO during training
- ✓ Use appropriate prior $\mathcal{N}(0,1)$

Common Pitfalls

- ✗ Insufficient model capacity
- ✗ Poor initialization
- ✗ Learning rate too high/low
- ✗ Not monitoring metrics
- ✗ Overfitting to training set

GAN Training Tips

- ✓ Balance D and G training
- ✓ Use label smoothing
- ✓ Add noise to discriminator inputs
- ✓ Use batch normalization
- ✓ Monitor mode collapse

Evaluation Metrics

- Inception Score (IS)
- Fréchet Inception Distance (FID)
- Reconstruction Error
- Visual Quality Assessment
- Diversity Metrics

Debugging Checklist

If VAE produces blurry images:

- Reduce KL weight (β -VAE)
- Increase latent dimension

If GAN won't converge:

- Balance D and G training
- Add noise to D inputs
- Check learning rates (try 0.0002)

General:

- Monitor metrics every epoch
- Visualize samples frequently

Visual Quality:

- Human evaluation (gold standard)
- Visual inspection of samples

Quantitative Metrics:

- **Inception Score (IS):** Diversity and quality
- **Fréchet Inception Distance (FID):** Distribution similarity
- **Reconstruction Error:** For VAE

Best Practice: Use multiple metrics, don't rely on one!

Common Pitfalls to Avoid

Don't:

- Start with complex architectures
- Ignore warning signs (mode collapse, divergence)
- Use only one metric for evaluation
- Skip hyperparameter tuning
- Train for too long without checking samples

Do:

- Start simple (MNIST before faces)
- Monitor training closely
- Save checkpoints frequently
- Compare multiple approaches
- Test on held-out validation set

Summary

Key Takeaways - Part 1

Core Concepts:

- Generative models learn to create new data
- Two main types: VAE and GAN
- Both learn from examples, generate new samples

Autoencoders:

- Compress and reconstruct
- Learn useful latent representations
- Foundation for more complex models

VAE:

- Probabilistic framework
- Smooth latent space
- Stable but blurry

GAN:

- Adversarial training
- Sharp realistic outputs
- Powerful but tricky to train

Key Takeaways - Part 3

Applications:

- Image generation and manipulation
- Data augmentation
- Style transfer
- Medical imaging
- Privacy-preserving synthetic data

Success Factors:

- Start simple
- Monitor carefully
- Balance stability and quality
- Use appropriate architecture for task

Advanced Topics:

- β -VAE, Conditional GANs
- StyleGAN, CycleGAN
- Diffusion Models

Resources:

- Interactive notebook for practice
- Try MNIST first
- Experiment with hyperparameters

Questions?